

Overview

This is the walk through of our AI agent module. Our goal was to design and implement multiple intelligent agents with increasing levels of complexity, ranging from simple rule-based behavior to advanced planning systems.

The module is built around a shared agent framework, where each agent inherits from the `BaseAgent` class and implements its own decision-making logic.

Across our files, we explore four main types of agents:

- A basic chasing agent
- An exploration-based agent
- A strategic combat agent
- And a planning-based agent using GOAP

Each one builds on the previous in terms of intelligence and behavior.

Modifications to base classes

AgentBase modification

We added a virtual `IsEnemy()` function to `AgentBase`.

By default, this returns false, meaning most agents are treated as friendly or neutral. Enemy agents can override it to return true.

This is useful because our AI can now identify hostile agents without using `dynamic_cast` or checking specific class types. Instead of asking 'what class is this?', the AI can simply ask 'is this an enemy?'

WorldBase modification

We also added `GetTrackedPlayerPosition()` to `WorldBase`.

This is an optional hook that allows a world to expose the position of a keyboard-controlled player, even if that player is not represented as a normal registered agent.

By default, it returns `std::nullopt`, so worlds that do not need it are unaffected.

EnemyAgent (Basic Behavior)

First, we have the `EnemyAgent`, which is our simplest AI. Its main goal is to chase the player using a greedy strategy.

- “It evaluates movement actions—up, down, left, right—and predicts the resulting position.”
- “It uses Manhattan distance to measure how far it is from the player.”
- “Each action is scored based on how much closer it gets to the target.”
- “Invalid moves like walls or out-of-bounds are heavily penalized.”

Overall, this agent always chooses the move that minimizes distance to the player, making it simple but effective.

LearningExplorerAgent (Exploration + Memory)

Next, we have the `LearningExplorerAgent`, which focuses on exploration instead of chasing.

- It keeps track of how many times each grid cell has been visited.
- It strongly prefers unvisited cells to explore new areas.
- It uses BFS to find the nearest unexplored location.
- It avoids:
 - walls
 - other agents
 - and revisiting the same tiles too often

It also includes anti-oscillation behavior, meaning it avoids going back and forth between the same two tiles.

This makes it much smarter for exploration tasks compared to a random or greedy agent.

SmartEnemyAgent (Strategic Combat AI)

After that, we implemented the SmartEnemyAgent, which introduces strategic decision-making.

- It prioritizes actions in this order:
 - Attack if adjacent
 - Chase the player
 - Explore if no target is available
- It uses line-of-sight checks to determine if it can directly pursue the player.
- If not, it uses BFS to find the shortest path.
- It also selects the best attack position around the player.

It even includes a 'hunt radius', meaning it won't chase the player if they're too far away—making the behavior more realistic.

This agent balances aggression and efficiency, making it significantly more advanced than the basic EnemyAgent.

TrailblazerAgent (GOAP Planning AI)

Finally, we have our most advanced agent: the TrailblazerAgent.

Key points:

- This agent uses GOAP—Goal-Oriented Action Planning.
- It represents the world state using bitmasks, including:
 - health
 - weapons
 - enemy presence
 - item visibility
- It dynamically selects goals such as:
 - healing when low HP
 - picking up weapons
 - attacking enemies
- It builds a plan by chaining actions that satisfy preconditions and effects.

It also combines planning with real-time reactions, like attacking immediately if an enemy is adjacent.

This makes the TrailblazerAgent adaptive, goal-driven, and the most intelligent agent in our module.

FetchAgent

We also implemented a FetchAgent, which is used in the Interactive World.

This agent has a simple but useful behavior: it moves back and forth between two points—an origin and a deposit location.

When it's empty, it travels to the origin to collect resources. When it's carrying something, it travels to the deposit point to drop it off.

It supports flexible behavior through callbacks, meaning we can customize what happens when it reaches each endpoint. For example, it can collect resources from a spawn or deposit them into a town hall.

The agent uses BFS pathfinding to move toward its target, stopping when it reaches a tile adjacent to it, since many endpoints are not directly walkable.

Integration

We integrated our AI agents into the main game system so they actually run during gameplay.

In Game.cpp, we included our AI agent files like SmartEnemyAgent and LearningExplorerAgent, allowing the game to create them. We also added the goblin sprite, which is used to visually represent these agents.

In the overworld, we added a LearningExplorerAgent at position (5, 5). This gives the world an AI-controlled character that explores independently.

To support this, we added a helper function AddLearningExplorerAgent in OverWorld.hpp, which keeps the spawning logic clean and reusable.

In the dungeon, we added a SmartEnemyAgent named “Goblin”. The game scans for a valid floor tile and places it there so it can move and interact properly.

We also integrated the FetchAgent into the Interactive World. This agent is designed to interact with that environment, handling tasks like retrieving or responding to objects in that world.

After the player takes a turn, the game loops through all non-player agents, calls SelectAction, and executes their chosen action. This is what makes all our agents behave dynamically in real time.

Finally, we updated rendering so our agents appear correctly. The explorer and the goblin are both displayed using the goblin sprite, making them visually distinct in their respective worlds.

Conclusion

To summarize:

- EnemyAgent → simple greedy chasing
- LearningExplorerAgent → exploration with memory
- SmartEnemyAgent → tactical combat decisions
- TrailblazerAgent → full planning with GOAP
- FetchAgent → BFS search

Each agent builds on the previous one, demonstrating increasing levels of AI sophistication.”